SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 1

Analytical Problem Solving

Name of the Student	MADDIRALA BALA MURALIKRISHNA
Reg. No	192325015

COURSE NAME: Deep Learning **COURSE CODE: MLA0403**

FACULTY : Dr . Arul Raj A.M

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

Assessment Tool Weightage: 30%

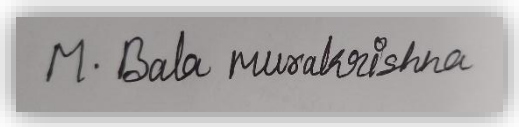
Duration: 30 minutes

Marks: 25

Code of Conduct:

I Maddirala Bala Muralikrishna (192325015) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.



MADDIRALA BALA MURALI KRISHNA

Signature of the student: **Name of the student:**

QUESTIONS: 05 Total Marks: 25

1. Learning Algorithms (Optimization Behavior)

A machine learning model is trained using gradient descent on a convex loss function. Initially, the learning rate is set very high, and later it is gradually decreased. Analyze how this strategy affects convergence, when it can outperform a constant learning rate, and its risks.

Answer:

Effect on convergence

- Early phase (high learning rate): large steps let the optimizer traverse the loss surface quickly, escaping flat regions and covering large distances toward the minimum in fewer iterations.
- Later phase (decayed learning rate): as the iterate nears the minimum, smaller steps prevent overshooting and allow fine-grained settling into the minimum, since near a minimum of a convex function the gradient itself becomes small and a large step relative to it causes oscillation.
- Overall trajectory: fast coarse progress followed by precise convergence — this is exactly the intuition behind learning-rate schedules (step decay, exponential decay, cosine annealing).

When this outperforms a constant learning rate

- If a constant rate is set high enough for fast early progress, it will typically be too large near the minimum and cause persistent oscillation or divergence — decay avoids this.
- If a constant rate is set low enough to be stable near the minimum, early convergence becomes very slow — decay avoids wasting iterations.
- For convex functions with a well-conditioned but large basin, decay strategies achieve a strictly better trade-off: the theoretical convergence rate of gradient descent with a decaying step size (e.g., $\eta_k \propto 1/k$) matches the $O(1/k)$ rate needed for convergence to the exact minimum, whereas a constant step size only guarantees convergence to a neighborhood of the minimum whose radius is proportional to the step size.
- Decay is especially beneficial when the loss surface has anisotropic curvature (steep in some directions, shallow in others), since a single constant rate cannot be optimal for both.

Risks

- Decaying too early or too aggressively can freeze progress before the model reaches a good region, effectively wasting the remaining training budget.
- Decaying too slowly delays the benefit and keeps oscillation risk for longer.
- The schedule introduces extra hyperparameters (decay rate, decay schedule, initial rate) that must themselves be tuned, adding complexity to the training pipeline.
- For non-convex losses (mentioned here only as contrast, since this problem is convex), a high initial rate is also useful for escaping poor local regions, but for a convex function the main benefit is purely the speed/precision trade-off described above.

Justification

For a convex, L-smooth loss, gradient descent with a fixed step size $\eta \leq 1/L$ converges, but the sub-optimality after k steps is bounded below by a term that depends on η and does not vanish unless $\eta \to 0$. A schedule that starts with a large η (close to 1/L) and lets $\eta_k \to 0$ as $k \to \infty$ (e.g., $\eta_k = \eta_0/(1+\lambda k)$) inherits the fast early contraction of a large step while still letting the sub-optimality gap shrink to zero, which is why decayed schedules dominate any single constant choice over the full trajectory.

2. Maximum Likelihood Estimation (MLE)

Given a dataset generated from a Gaussian distribution with unknown mean μ and known variance σ², derive the MLE for μ and explain how the estimate changes with outliers and with increasing sample size.

Answer:

Derivation of the MLE for μ

For i.i.d. samples $x_1, x_2, \ldots, x_n \sim N(\mu, \sigma^2)$, the likelihood is:

$$L(\mu) = \prod_{i=1}^{n} (1 / \sqrt{(2\pi\sigma^2)}) \cdot \exp(-(x_i - \mu)^2 / (2\sigma^2))$$

Taking the log-likelihood:

$$\ell(\mu) = -(n/2) \log(2\pi\sigma^2) - (1/2\sigma^2) \Sigma_i (x_i - \mu)^2$$

Differentiating with respect to μ and setting the derivative to zero:

$$d\ell/d\mu = (1/\sigma^2) \Sigma_i (x_i - \mu) = 0 \quad \rightarrow \quad \Sigma_i x_i = n\mu$$

$$\mathbf{\hat{\mu}\_MLE = (1/n) \Sigma_{i=1}^{n} x_i \quad (the\ sample\ mean)}$$

The second derivative $-n/\sigma^2$ is negative, confirming this critical point is the maximum.

Effect of outliers

- The sample mean is a linear, unbounded function of every observation, so a single extreme outlier x_out shifts $\hat{\mu}$ by $(x_{out} - \hat{\mu})/n$ — the shift does not vanish as the outlier magnitude grows, it grows with it.
- Formally, the mean has an influence function that is unbounded and a breakdown point of 0% (a single corrupted point can move the estimate arbitrarily far), so MLE under the Gaussian assumption is highly sensitive to outliers.

Effect of increasing sample size

- By the Law of Large Numbers, $\hat{\mu} \to \mu$ (converges to the true mean) as $n \to \infty$, since it is an unbiased estimator: $E[\hat{\mu}] = \mu$ for every n.
- Its variance decreases as $Var(\hat{\mu}) = \sigma^2/n$, so the estimator becomes increasingly concentrated around the true mean — by the Central Limit Theorem, $\hat{\mu}$ is asymptotically Normal$(\mu, \sigma^2/n)$.

Implication for robustness of MLE

MLE under the Gaussian model is efficient (achieves the Cramer-Rao lower bound) and consistent, but it is not robust: its optimality relies on the data truly being Gaussian and free of gross contamination. In practice this motivates robust alternatives such as the median, trimmed mean, or M-estimators.

3. Building a Machine Learning Algorithm (Model Selection)

You are designing a classification algorithm for a dataset with 10,000 features but only 500 training samples. Analyze the challenges and propose a justified pipeline to mitigate overfitting.

Answer:

Challenges

- Curse of dimensionality: with $p = 10{,}000 \gg n = 500$, the feature space is so sparse that any model can find spurious combinations of features that perfectly separate the training data without capturing any real signal.
- Severe overfitting risk: a model with enough parameters (e.g., a linear classifier with 10,000 weights) can trivially memorize 500 points, giving near-perfect training accuracy but poor generalization.

- Ill-conditioned / singular problems: with p > n, the feature covariance matrix is singular, so estimators that require its inverse (e.g., ordinary least squares, LDA) are not well-defined without modification.
- Noise accumulation: many of the 10,000 features are likely irrelevant or redundant; their combined noise can overwhelm the signal from the truly predictive features.
- Unreliable validation: with only 500 samples, held-out validation/test splits have high variance, making model comparison noisy unless resampling methods are used.

Proposed pipeline

- Step 1 — Filter-based feature screening: rank features by a fast univariate criterion (mutual information, ANOVA F-score, or correlation with the label) and drop clearly uninformative features before modelling, cheaply reducing dimensionality without touching model parameters.
- Step 2 — Dimensionality reduction / embedded selection: apply PCA (unsupervised) or, preferably for classification, L1 (Lasso) regularized logistic regression or Elastic Net, which perform embedded feature selection by driving irrelevant feature weights to exactly zero while fitting the model.
- Step 3 — Strong regularization on the chosen model: combine L2 (ridge) with L1, or use a max-margin classifier such as an SVM with an RBF or linear kernel and a tuned regularization parameter C, since regularization directly controls the effective capacity of the model relative to the tiny sample size.
- Step 4 — Cross-validation (k-fold, e.g., k = 5 or 10) instead of a single train/test split, to get a stable estimate of generalization performance given the small n, and to tune hyperparameters (regularization strength, number of retained components) without leaking test information.
- Step 5 — Ensembling / simplicity bias: prefer simpler models (regularized linear models, shallow trees with bagging) over high-capacity models (deep networks), since with n = 500 the bias-variance trade-off strongly favors low-variance estimators.

Why this mitigates overfitting

- Feature selection/reduction directly shrinks the effective dimensionality, moving the problem closer to the regime where n is comparable to (or larger than) the number of active features.
- Regularization constrains the hypothesis space, so even if some noise features slip through, their weights are shrunk toward zero and cannot dominate predictions.
- Cross-validation ensures the model's hyperparameters are chosen to generalize, not simply to minimize error on one particular split of a small dataset.

Together these steps address the $p \gg n$ regime by reducing effective capacity at every stage of the pipeline rather than relying on any single fix.

4. Neural Networks (Multilayer Perceptron - MLP)

Consider an MLP with one hidden layer using sigmoid activations. Explain why adding more hidden neurons increases representational power, and why this does not always improve performance.

Answer:

Why more hidden neurons increase representational power

- The Universal Approximation Theorem states that an MLP with a single hidden layer of sigmoid units can approximate any continuous function on a compact domain to arbitrary accuracy, provided the hidden layer is wide enough — more neurons means more basis functions (shifted, scaled sigmoids) available to combine.
- Each hidden neuron effectively contributes one soft 'step' function (a sigmoid) positioned by its weights and bias; the output layer linearly combines these steps. More neurons means more such steps can be summed, allowing the network to sculpt increasingly complex, non-linear decision boundaries or regression surfaces.
- Formally, the hypothesis space (the set of functions the network can represent) grows monotonically with the number of hidden units, since any function representable by h hidden units is also representable by h+1 units (the extra unit's contribution can be set to zero).

Why this does not always improve performance

- Overfitting: with enough neurons, the network has enough free parameters to fit not just the true underlying pattern but also the noise in the finite training set, which hurts performance on unseen data even though training error keeps decreasing.
- Bias-variance trade-off: increasing width lowers bias (the model can represent more complex functions) but increases variance (small changes in the training set lead to very different learned functions), and beyond some point the variance increase outweighs the bias reduction.
- Optimization difficulty: a much larger parameter space is harder to optimize with a fixed amount of data and gradient descent iterations — the network may settle in a poor region of the loss surface, or need far more data/epochs to actually realize its extra capacity.

Conclusion

Representational capacity and generalization performance are not the same axis: capacity is a ceiling on what the network could represent, while generalization depends on whether the training data and regularization are sufficient to guide the network toward the true function rather than an overfit one. The practical implication is that hidden-layer width should be chosen relative to dataset size and paired with regularization (weight decay, dropout, early stopping) rather than maximized in isolation.

5. Backpropagation, SGD & Curse of Dimensionality

A deep neural network is trained using SGD on a very high-dimensional dataset. Analyze the impact on gradient updates, convergence speed, and generalization, and propose techniques to overcome these issues.

Answer:

Impact on gradient updates in backpropagation

- In very high dimensions, gradients computed via backpropagation are estimated from finite mini-batches, so each individual gradient component (one per weight) is a noisy estimate; with many more parameters than can be reliably estimated from a batch, this noise is spread across a much larger vector, making individual gradient directions less reliable.
- Vanishing/exploding gradients become more likely as depth and input dimensionality grow, since the chain rule multiplies many partial derivatives together, and in high-dimensional layers the scale of these products is harder to control without careful initialization/normalization.
- Sparse or highly correlated input features (common in high-dimensional data) can cause certain weight directions to receive very small, uninformative gradient signals while others dominate, leading to uneven learning across parameters.

Impact on convergence speed of SGD

- The curse of dimensionality means data becomes sparse relative to the volume of the input space, so the loss surface can have many flat regions, saddle points, and narrow ravines that slow convergence, since gradient descent moves slowly along directions of small curvature.
- More parameters generally mean the optimizer needs more iterations (and often more data) to traverse the loss landscape, and the mini-batch gradient noise (variance) does not automatically shrink just because dimensionality increased, so more steps are needed for the noise to average out.
- Ill-conditioning (very different curvatures along different directions) becomes more likely in high dimensions, forcing a smaller learning rate for stability, which further slows convergence.

Impact on model generalization

- With high input dimensionality but limited samples, the effective sample density per unit volume of feature space collapses exponentially (the core curse-of-dimensionality effect), so the network cannot observe enough examples to reliably estimate the true underlying function everywhere it needs to generalize.
- This increases the risk that SGD converges to a solution that fits noise or spurious high-dimensional correlations, producing a larger train-test performance gap.

Techniques to overcome these issues (at least two, justified)

- Dimensionality reduction / representation learning (e.g., PCA, autoencoders, or learned embeddings): reduces the effective input dimensionality before or within the network, directly counteracting data sparsity and reducing the number of directions gradients must estimate reliably.
- Batch normalization: normalizes layer inputs during training, which stabilizes the scale of activations and gradients layer-to-layer, mitigates vanishing/exploding gradients, and allows the use of larger, more effective learning rates — directly addressing both the gradient-update and convergence-speed issues.
- Regularization (L2 weight decay, dropout): constrains the effective capacity of the network so that it cannot simply memorize high-dimensional noise, directly improving generalization even when dimensionality is high relative to sample size.
- Adaptive optimizers (Adam, RMSProp): rescale each parameter's update by a running estimate of its gradient magnitude, which helps when different dimensions have very different gradient scales (a common symptom of high-dimensional, ill-conditioned problems), improving convergence speed over plain SGD.

Combining a dimensionality-reduction/regularization strategy (to address generalization and effective capacity) with normalization/adaptive optimization (to address gradient stability and convergence speed) targets all three of the analyzed effects rather than any single one in isolation.